

Тема: Функції в мові JavaScript**План лекції**

1. Поняття функцій.
2. Створення функцій.
3. Використання аргументів.
4. Рекурсивні функції.

Зміст лекції

Організація одній або безліч функцій відноситься до однієї з цілей всіх програм на JavaScript. У найпростішому випадку сценарій може читати або приймати дані, виконувати певні операції на наборі даних або відображати і посилати дані.

1. Поняття функцій

JavaScript-функція — це просто сценарій, що відокремлений від решти частини програми і має своє ім'я. Використовуючи це ім'я, інший сценарій може викликати його у будь-який момент і скільки завгодно раз. Деякі мови програмування, наприклад, C/C++ або Java, також використовують функції, тоді як інші мови мають ті ж самі семантичні одиниці, але називають їх методами, процедурами або підпрограмами. Всі вони роблять в основному ті ж самі речі, але мають деякі відмінності.

Функції служать одній меті — розділенню завдань, призначених для виконання в одній програмі. Можна вважати функцію повідомленням JavaScript про необхідність виконання конкретного списку зв'язаних команд і видачі сигналу, як тільки це буде зроблено.

При виклику функції можна передати значення, звані аргументами (arguments). Аргументи можна використовувати як змінних (variables) в межах одного блоку операторів. Як тільки дані привласнені змінною, стає можливою обробка даних або використання їх в обчисленнях.

2. Створення функцій

Нижчих приводиться синтаксис оголошення функції в JavaScript:

function ім'я_функції (аргумент 1) (... , аргументи) {оператори}

Ключовими словами ***function*** використовується для визначення функції з ім'ям ***ім'я_функції***, яке служить як ***ідентифікатор*** для набору ***операторів***, які знаходяться у фігурних дужках. Імена аргументів, що зберігають значення які передаються у функцію, знаходяться у круглих дужках і виділяються один від одного комами.

З погляду програмування, аргументи - змінні, яким можна привласнювати літеральні значення, інші змінні або об'єкти, передавані у функцію при виклику. Навіть у разі повної відсутності аргументів все одно необхідно після імені Функції вказувати пару круглих дужок, тим самим позначаючи ***ідентифікатор*** як функцію.

Оператори в тілі функції виконуються при кожному виклику функції. Для підвищення легкості читання рядка всередині блоку операторів вирівнюються.

Де оголошувати функції

Теоретично функцію можна оголошувати в будь-якому місці в межах розділу `<script>`. Існує єдине обмеження - неможна оголошувати функцію в межах Іншої Функції або структури, що керує. Проте, майте на увазі, одні блоки HTML-документа завантажуються раніше інших, так само як і будь-які сценарії, вбудовані в ці HTML-блоки. У зв'язку з цим рекомендується оголошувати функції HTML-документа в розділі `<head>`. Оголошення всіх функцій в цьому розділі гарантує, що функції будуть доступні, якщо якомусь сценарію треба буде викликати їх негайно.

У лістингу 2.3.1 показана функція **`defaultColors()`**, яку оголошено усередині розділу `<head>`. Далі ця функція викликається в розділі `<body>` документа.

Лістинг 2.3.1 Оголошення Функції в розділі <head>.

`<html>`

```

<head>
  <title> Використання функції: </title>
  <script type="text/javascript">
    function defaultColors() {
      document.writeln("В середині defaultColor());
      document.fgColor = "white";
      document.bgColor = "green";
    }
  </script>
</head>
<body>
  <script type="text/javascript">
    document.write("Функції - сценарії очікують Виконання!");
    defaultColors();
    document.write(" Виконання закінчено.");
  </script>
</body>
</html>

```

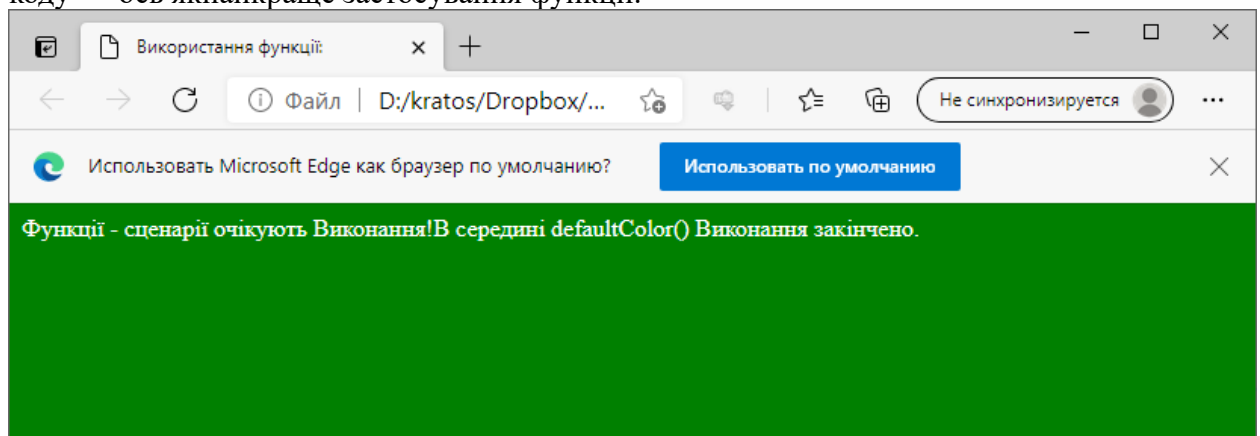
Виклик Функції

Лістинг 2.3.1 демонструє виклик Функції defaultColors() з іншого розділу сценарію. Це приклад виклику функції без аргументів. Після повного завантаження HTML-документа функція зберігається в пам'яті й знаходиться там "в стані бойової готовності". Вона НЕ виконуватиметься, поки головний розділ сценарію НЕ викличе її наступний оператором:

```
defaultColors()
```

У цій точці програма відразу ж переходить на перший рядок функції defaultColors(). Після виконання всіх трьох рядків програма повертається назад до місця, в якій вона покинула головний розділ, й продовжує роботу там.

Таким чином виходить такий же ефект, як і при приміщенні всього коду функції безпосередньо в конкретну позицію програми. Тепер, коли функції привласнено ім'я, все, що потрібно буде зробити для виконання тих же інструкцій — використовувати її ім'я знову. Результати описаного процесу показані на мал. 2.3.1. Багаторазове використання якогось коду — ось якнайкраще застосування функції.



МАЛЮНОК 2.3.1. Повторне використання коду.

Примітка

На відміну від властивості рядка fontcolor, властивості документа fgcolor и bgcolor можна змінювати без оновлення або перезавантаження сторінки в браузері.

3. Використання аргументів

Функції з аргументами не менш корисні. Аргументи дають можливість багато разів використовувати функцію у різних завданнях для досягнення одних і тих же цілей. Наприклад, можна створити функцію, необхідність в якій виникає багато разів в ході виконання програми; проте при цьому кожного разу при виклику функції потрібно змінювати одне із значень усередині функції.

Один із способів вирішення згаданої проблеми без задіяння механізму аргументів зв'язаний із застосуванням глобальних змінних, які можуть змінюватися як зовні, так і усередині функції. Програма стане заплутаною, якщо ті ж самі змінні використовуватимуться усередині більш ніж одній функції. Вигідно створити функцію з аргументом, що приймає всі необхідні значення.

Можна використовувати функції, що повертають значення, в звичайних виразах. Деякі функції повертають результат набору обчислень, тоді як інші — логічне значення, що повідомляє про те, чи все в порядку (див. лістинг 2.3.2).

Лістинг 2.3.2. Функції, що повертають значення, які можна використовувати у виразах.

```
<html>
<head>
  <title> Функції </title>
  <Script type="text/javascript">
    function isPhone(aString) {
      var aChar = null;
      var status = true;
      if (aString.length != 13) {
        status = false;
      } else {
        for (var i = 0; i <= 12; i++) {
          aChar = aString.charAt(i);
          if (i == 0 && aChar == "(") {
            continue;
          } else {
            if (i == 4 && aChar == ")") {
              continue;
            } else {
              if (i == 8 && aChar == "-") {
                continue;
              } else {
                if (parseInt(aChar, 10) >= 0 && parseInt(aChar, 1
0) <= 9) { continue; }

                else {
                  status = false;
                  break;
                }
              }
            }
          }
        }
      }
      return (status);
    }
  }
</script>
```

```

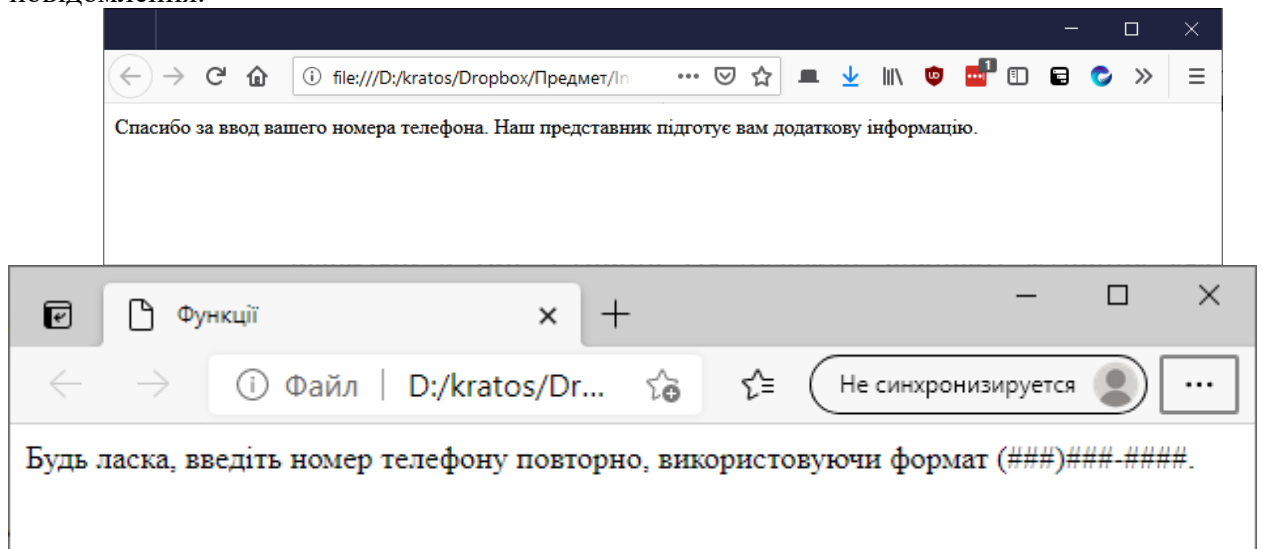
</head>

<body>
  <script type="text/javascript">
    var userInput = "(800)555-12122";
    if (isPhone(userInput)) {
      document.writeln("Дякую за введення вашого номера телефону.");
      document.writeln("Наш представник підготує вам ");
      document.writeln(" додаткову інформацію.");
    } else {
      document.writeln("Будь ласка, введіть номер телефону повторно, ")
;
      document.writeln("використовуючи формат (###)###-####.");
    }

  </script>
</body>
</html>

```

Виклик функції `isPhone(userInput)` застосовується як умовний вираз в операторові `if`. Функція `sPhone()` повертає логічне значення. Повернене значення дає знати головній програмі, чи в коректному форматі був введений телефонний номер. Це корисно для перевірки достовірності даних, що вводяться користувачами в HTML-форму. На мал. 2.3.2 видно, що телефонний номер був введений коректним, на що указує відповідне повідомлення.



МАЛЮНОК 2.3.2. Використання Функції в якості умовного виразу.

Зміна кількості аргументів

Для функції встановлюється певна кількість аргументів. Не дивлячись на те що використання в точності оголошеної кількості аргументів вважається хорошим стилем програмування, іноді виникає необхідність установки іншої кількості аргументів. Іноді це потрібно при виклику функції, яка кожного разу використовує одне і те ж значення параметра і змінює його тільки в спеціальних випадках.

Якщо аргументи не передаються, слід використовувати їх значення за умовчанням, визначені в тілі функції. Це дозволяє викликати функцію без аргументів або визначати значення, відмінне від значення за умовчанням. У лістингу 2.3.3 створюється функція для відображення типового вітання, що з'являється, коли користувач заходить на Web-сторінку.

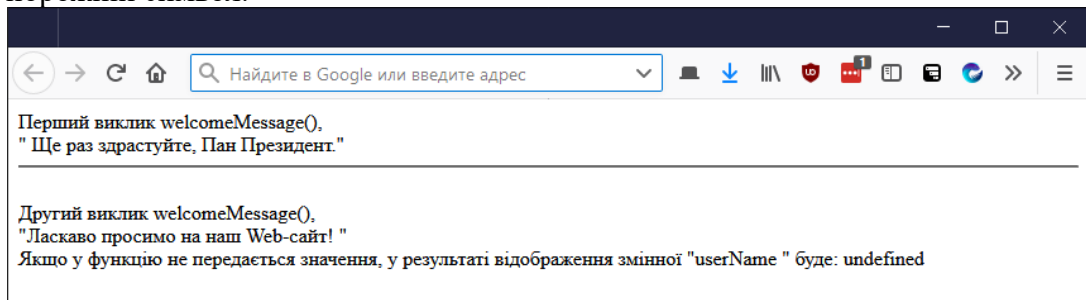
Наступна програма відображає одне з двох повідомлень, залежно від того, чи відоме їй ім'я відвідувача. Якщо `userName` не рівне `null`, це означає, що змінна визначена. Таке можливе тільки якщо у функцію передається значення, наприклад, Пан Президент. Якщо аргумент функції рівний `null`, змінна у вітальному повідомленні взагалі не використовується.

Лістинг 2.3.3. Функція, що приймає один аргумент або що не приймає жодного аргументу.

```
<html>
<head>
  <title> Функції </title>
  <script type="text/javascript">
    // userName є необов'язкова
    function welcomeMessage(userName) {
      if (userName != null) {
        document.writeln("\n Ще раз здрастуйте, " + userName + ".\n");
      }
      else {
        document.writeln("\nЛаскаво просимо на наш Web-сайт! \n");
        document.write("<br>Якщо у функцію не передається значення, ");
        document.writeln(" у результаті відображення ");
        document.writeln(" змінної \"userName \" буде: ");
        document.writeln(userName);
      }
    }
  </script>
</head>

<body>
  <script type="text/javascript">
    document.writeln("Перший виклик welcomeMessage(), <br>");
    welcomeMessage("Пан Президент");
    document.writeln("<HR> <br> Другий виклик welcomeMessage(), <br>");
    welcomeMessage();
  </script>
</body>
</html>
```

Як показано на мал 2.3.3, її використання приводить до відображення небажаних даних. Залежно від конкретного браузера, небажані дані відображатимуться як `undefined` або як порожній символ.



МАЛЮНОК 2.3.3. Результат використання аргументів.

ПОРАДА

На теперішній час JavaScript підтримує операцію **typeof**, яка забезпечує перевірку, чи є `userName` невизначеним або рівним `null`. Це ідеальний спосіб перевірки.

Ще один випадок пов'язаний з передачею у функцію більшої кількості аргументів, чим було визначено в оголошенні. Додаткові значення при цьому не втрачаються, а зберігаються в масиві `arguments`, який є властивістю кожної функції. Всі аргументи, збережені в масиві, можуть витягувати в межах тіла функції. Наприклад, перший аргумент, переданий у функцію `welcomeMessage`, можна отримати так:

firstArg = welcomeMessage.arguments[0]

Елементи масивів JavaScript нумеруються, починаючи з 0. Другий елемент в масиві матиме індекс 1 і так далі

Для обчислення загальної кількості переданих аргументів можна записати такий рядок:

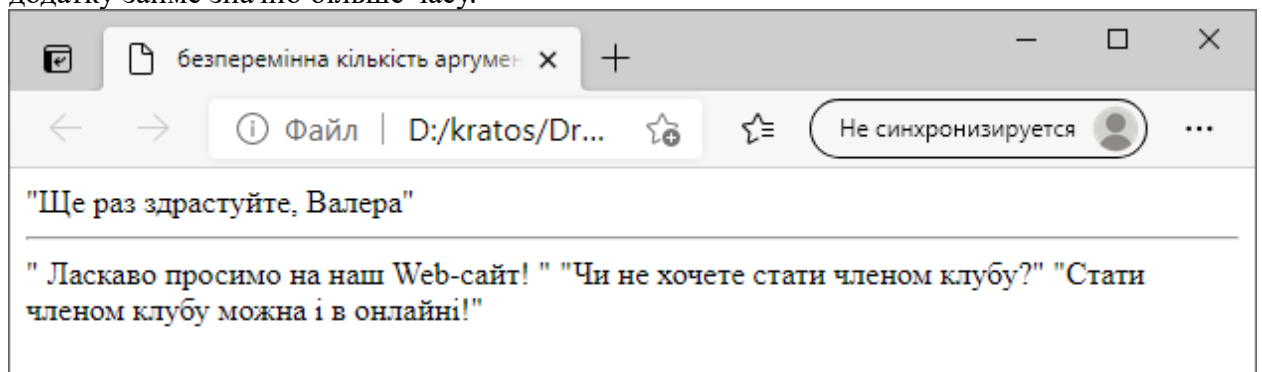
`numArgs = welcomeMessage.arguments.length`

У лістингу 2.3.4 демонструються всі згадані можливості, при цьому в `welcomeMessage()` передається змінна кількість аргументів. В даному випадку функція `welcomeMessage()` набуває наступного вигляду:

`welcomeMessage( [имяКористувача] [, додатковеПовідомлення1] [,додатковеПовідомлення 2]. ..)` Зверніть увагу, що в лістингу 2.3.4 ситуація, при якій `userName` невідомо, проте додаткові повідомлення повинні відображатися, все ще підтримується. Змінною `userName2` привласнено значення `null` для того, щоб заповнити перший елемент в масиві аргументів так, щоб повідомлення не розглядалося як ім'я користувача. На мал. 2.3.4 показано, що зі всього цього вийшло.

ПОРАДА

При колективній розробці всі складні сценарії необхідно документувати, щоб все, написане особисто вами, змогли зрозуміти інші. Без простих і наочних коментарів читання додатку займе значно більше часу.



МАЛЮНОК 2.3.4. Передача у функцію безлічі аргументів для відображення безлічі Повідомлень.

Лістинг 2.3.4. У функцію можна передати змінну кількість аргументів.

```
<html>
<head>
  <title> безперемінна кількість аргументів у функції </title>
  <script type="text/javascript">
    // При визначенні welcomeMessage скористайтеся таким синтаксисом:
    // welcomeMessage ([userName] [, extraMessage1] [, extraMessage2].
    ..)
    function welcomeMessage(userName) {
      if (userName != null) {
        document.writeln("\nЩе раз здрастуйте, " + userName + "\n");
      } else {
        document.writeln("\n Ласкаво просимо на наш Web-сайт! \n");
        numArgs = welcomeMessage.arguments.length;
```

```

        // Якщо у функцію, окрім userName, передаються додаткові а
ргументи, відобразити кожен з них
        if (numArgs > 1) {
            for (var i = 1; i < numArgs; i++) {
                document.writeln("\n" + welcomeMessage.arguments[i
] + "\n");
            }
        }
    }
}
</script>
</head>

<body>
    <script type="text/javascript">
        var userName = "Валера";
        extraMsg = "Давненько не зустрічалися!";
        var userName2 = null;
        var extraMsg1 = "Чи не хочете стати членом клубу?";
        var extraMsg2 = "Стати членом клубу можна і в онлайні!";
        welcomeMessage(userName, extraMsg);
        document.writeln("<hr>");
        welcomeMessage(userName2, extraMsg1, extraMsg2)
    </script>
</body>
</html>

```

4. Рекурсивні функції.

Функції JavaScript можуть бути рекурсивними, тобто викликати самі себе. Обчислення факторіалу — кращий спосіб демонстрації рекурсивного процесу. Зважаючи на те що JavaScript не має вбудованої операції факторіалу, їх доведеться обчислювати самостійно, причому з використанням механізму рекурсивних функцій. Для знаходження факторіалу будь-якого позитивного цілого числа n потрібно просто знайти твір всіх цілих чисел від 1 до n . Наприклад, факторіал числа 6 (записується як $6!$) обчислюється таким чином:

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

$7!$ обчислюється так:

$$7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 5040$$

На основі порівняння цих двох виразів можна записати загальну формулу, яка використовуватиметься в створюваній функції. Звернете увагу, що $7!$ ні що інше як $7 \times 6!$. Для будь-якого позитивного цілого числа n справедливий запис $n! = n \times (n - 1)!$. Перша ітерація виглядає так:

$$7! = 7 \times (7 - 1)!$$

$$= 7 \times 6!$$

Тепер необхідно зупинитися і обчислити $6!$. Поступаючи аналогічним чином ще шість разів, отримаємо остаточне рішення. Проте набагато ефективніше скористатися рекурсивною функцією, наприклад, такий:

```

function getFactorial(n) {
    var result;
    if (n > 0) {
        result = n * getFactorial(n - 1);
    }
}

```



```

        else if (n == 0) {
            result = 1;
        }
        else {
            result = null;
        }
        return (result)
    }

```

Функція спочатку порівнює n з нулем. Якщо вираз $n > 0$ істинно, функція множить n на результат, повернений з виклику цієї ж функції з іншим аргументом, в даному випадку, $n - 1$. Вкладені виклики функції `getFactorial()` продовжують чекати, поки n не стане рівним 0. У цей момент найбільш вкладене виконання функції завершується і повертає перше значення. JavaScript повертається назад і завершує кожну вкладену функцію, поки не досягне першого виклику `getFactorial`, після чого вийде остаточний результат.

Розробка корисних функцій — один з найбільш істотних і цікавих аспектів програмування. Після того, як функція перевірена у дії, варто повернутися і підправити її, спробувати зробити гнучкішою. Після не забудьте переконатися, що функція все ще вирішує задану мету і не робить нічого несподіваного.

Контрольні питання.

1. **Оголошення функції:** Як створити функцію за допомогою ключового слова `function`? Опишіть синтаксис: ім'я, параметри в дужках, тіло функції.
2. **Виклик функції:** Як викликати функцію з іншого місця коду? Як передати їй аргументи?
3. **Повернення значення:** Як функція повертає результат за допомогою оператора `return`? Що поверне функція, якщо `return` відсутній?
4. **Параметри функції:** Чи обов'язково в JavaScript передавати при виклику стільки ж аргументів, скільки вказано параметрів в оголошенні? Що буде зі значеннями відсутніх аргументів?
5. **Об'єкт `arguments`:** Що таке масив `arguments` всередині функції? Як за його допомогою отримати доступ до всіх переданих аргументів, навіть якщо вони не були оголошені як параметри?
6. **Властивість `arguments.length`:** Як дізнатися кількість фактично переданих аргументів у функцію під час її виконання?
7. **Рекурсія:** Що таке рекурсивна функція? Наведіть приклад (наприклад, обчислення факторіалу), де функція викликає сама себе.
8. **Область видимості змінних у функціях:** Чи видно змінні, оголошені всередині функції (через `var`), зовні цієї функції? Чи має функція доступ до глобальних змінних?
9. **Перетворення типів:** Як працює функція `parseInt()`? Що вона поверне, якщо рядок починається з цифр, а закінчується літерами (наприклад, "123abc")?
10. **Передача значень:** Що відбувається, якщо у функцію передати `null` замість очікуваного значення? Як це можна перевірити всередині функції?
11. **Приклад з телефонним номером:** У лекції розглядався приклад функції валідації номера. Які перевірки (довжина, конкретні символи на певних позиціях) виконувала ця функція?
12. **Вкладені виклики:** Чи можна викликати одну функцію всередині іншої або передавати результат виконання однієї функції як аргумент для іншої?